

signus 변경분

바뀐 파일은 통째로 실었습니다 — 그 파일 쪽만 새로 쓰면 됩니다

기준점 2026-08-10 (801d39a+수정중) → 2026-08-13 · ebe6e5e+수정중 | 1개 파일 · +102 / -58 줄 · 코드 7쪽 | 새 코드북 71쪽

바뀐 내용

- 버스트 탐지가 워터폴(폭포수) 방식이 됐습니다. 눈으로 스펙트로그램을 보듯 주파수 칸마다 바닥을 따로 재므로, 전체 대역 기준으로는 잡음에 묻히는 좁은 대역의 약한 버스트(광대역 0dB급)도 이제 잡습니다.
- 잡음뿐인 파일, 연속 신호, 다중톤 FSK는 예전처럼 파일 전체 한 구간으로 나옵니다. 768조합 신규 대조에서 회귀 0건, 오경보 0건 — 광대역 쪽은 오히려 예전이 놓치던 59개까지 새로 잡습니다.
- 버스트 경계가 64샘플 격자에 정렬돼 같은 신호면 항상 같은 경계가 나오고, 가장자리 잡음을 덜 물어 복조 품질(MER)이 좋아집니다. 새로 칠 파일은 dsp.py 하나입니다.

□ `signus/dsp.py`

+102

-58

2쪽

코드북 15쪽

초록 = 새로 쓸 줄 (오른쪽 숫자가 새 줄번호) · 색 없는 줄 = 그대로 · **붉은 점선** = 그 자리에서 몇 줄 사라짐 · **↵** = 윗줄에 붙는 이어짐 (원래 한 줄이니 붙여서 입력)

signus/dsp.py +102 -58 · 코드북 15쪽 · 새로 쓸 줄 8, 66-68, 74, 76-104, 106-143, 150-151, 156-160, 164-165, 167-168, 170-176, 183-185, 189-197

```

1 1 """Receiver DSP stages for blind PSK/QAM demodulation. Vectorized; per the house
2 2 anti-shared-bug rule this imports only constellation reference data, never gen.py."""
3 3
4 4 from fractions import Fraction
5 5
6 6 import numpy as np
7 7 from scipy.ndimage import uniform_filter1d
8 + from scipy.signal import get_window, hilbert, resample_poly, stft, welch
9 9
10 10 from ._accel import dd_carrier
11 11 from .constellations import ideal_points
12 12
13 13
14 14 def analytic(x: np.ndarray) -> np.ndarray:
15 15     """Complex passthrough as complex128; real input -> Hilbert analytic signal. Non-finite
16 16     samples are zeroed first: a single NaN/Inf otherwise spreads through every FFT (and Inf
17 17     overflows on the  $|x|^2$  the estimators take)."""
18 18     x = np.nan_to_num(x, posinf=0.0, neginf=0.0)
19 19     x = np.where(np.abs(x) > 1e150, 0.0, x) # a finite spike that overflows  $|x|^2$  is corrupt t
    00
20 20     if np.iscomplexobj(x):
21 21         return x.astype(np.complex128)
22 22     return hilbert(np.asarray(x, dtype=np.float64)).astype(np.complex128)
23 23
24 24
25 25 def _parab(ydb: np.ndarray, k: int) -> float:
26 26     """Sub-bin peak offset from a 3-point parabola fit (inputs already in dB)."""
27 27     if k <= 0 or k >= ydb.size - 1:
28 28         return 0.0
29 29     a, b, c = ydb[k - 1], ydb[k], ydb[k + 1]
30 30     d = a - 2 * b + c
31 31     # clamp to +-half a bin: a true peak's sub-bin offset cannot exceed that, but a near-flat
32 32     # (d~0) or non-max triple can explode the ratio and drive baud negative -> a resample crash
    .
33 33     return float(np.clip(0.5 * (a - c) / d, -0.5, 0.5)) if d != 0 else 0.0
34 34
35 35
36 36 def _kmeans1d(a: np.ndarray, k: int, iters: int = 50) -> tuple[np.ndarray, np.ndarray, float]:
37 37     """1-D k-means on quantile-seeded centers; return (centers, labels, rms spread).
38 38     Assignment is a running min over the k centers (no (N, k) broadcast temporary);
39 39     strict '<' keeps the lowest-index center on ties, so labels are identical to a
40 40     broadcast argmin. Shared by the classify amplitude-ring test and the FSK level demod."""
41 41     c = np.quantile(a, (np.arange(k) + 0.5) / k)
42 42     lab = np.zeros(a.size, dtype=int)
43 43     for _ in range(iters):
44 44         d = np.abs(a - c[0])
45 45         lab = np.zeros(a.size, dtype=int)
46 46         for j in range(1, k):
47 47             dj = np.abs(a - c[j])
48 48             closer = dj < d
49 49             lab[closer] = j
50 50             d[closer] = dj[closer]
51 51         nc = np.array([a[lab == j].mean() if np.any(lab == j) else c[j] for j in range(k)])
52 52         if np.allclose(nc, c):
53 53             break
54 54         c = nc
55 55     return c, lab, float(np.sqrt(np.mean((a - c[lab]) ** 2)))
56 56
57 57

```

```

58 58 # --- burst detection -----
59 59
60 60 _SPIKE_PAR = 60.0 # raw peak-to-mean power above which a candidate run is an impulse smear, no
    L L t a
61 61 # burst. Modulated bursts sit low: constant-envelope FSK ~1, RRC-shaped PSK/Q
    L L AM
62 62 # peaks ~6-12, high-order QAM tails to ~20; a single-sample impulse smeared o
    L L ver
63 63 # a ~win-wide run scores ~win (hundreds+). 60 clears every real mod with marg
    L L in.
64 64
65 65
66 + def _otsu(v: np.ndarray, bins: int = 128) -> float:
67 + """Otsu split threshold of a 1-D value distribution (log powers, column scores)."""
68 + hist, edges = np.histogram(v, bins=bins)
78 69 centers = (edges[:-1] + edges[1:]) / 2
79 70 w = np.cumsum(hist).astype(float)
80 71 m = np.cumsum(hist * centers)
81 72 mb = m / np.where(w > 0, w, 1)
82 73 mf = (m[-1] - m) / np.where(w[-1] - w > 0, w[-1] - w, 1)
74 + return float(centers[int(np.argmax(w * (w[-1] - w) * (mb - mf) ** 2))])
87 75
76 +
77 + def _cell_power(x: np.ndarray, fs: float, nperseg: int = 256) -> tuple[np.ndarray, int, int]:
78 + """Waterfall cell power for detection: (P[fbin, col], hop, nperseg). nperseg
79 + auto-shrinks on short records; shared core for find_bursts (and, later, survey)."""
80 + nperseg = int(min(nperseg, max(64, 1 << int(np.log2(max(x.size // 2, 64)))))
81 + hop = nperseg // 2
82 + win = get_window("blackmanharris", nperseg)
83 + _, _, z = stft(x, fs=fs, window=win, nperseg=nperseg, noverlap=nperseg - hop,
84 + return_onesided=False, boundary=None, padded=False)
85 + return np.abs(z) ** 2, hop, nperseg
86 +
87 +
88 + def find_bursts(x: np.ndarray, fs: float) -> list[tuple[int, int]]:
89 + """Waterfall (cell-level) burst detector. Per-bin noise floors give the same
90 + narrowband processing gain the operator's eye gets on a spectrogram (a burst at
91 + wideband 0 dB is +12 dB per cell when it occupies 1/16 of the band); column
92 + scores then drive the run/merge/guard machinery on the time axis.
93 + Returns bursts in time order, or [(0, size)] when nothing stands out."""
94 + n = x.size
95 + pw = np.abs(x) ** 2
96 + # A burst is a TIME-ENERGY event. A continuous signal that merely shuffles its spectrum
97 + # (multi-tone FSK) lights different cells per column and once shattered into 78 phantom
98 + # bursts -- but its wideband envelope is FLAT. Envelope Otsu separation measured 0.014-
99 + # 0.052 decades on continuous FSK vs >=0.30 on every real burst regime: nothing turns
100 + # on or off -> the whole record is the burst.
101 + lp = np.log10(uniform_filter1d(pw, max(64, n // 1000)) + 1e-20)
102 + e_thr = _otsu(lp)
103 + e_lo, e_hi = lp[lp < e_thr], lp[lp >= e_thr]
104 + if not e_lo.size or not e_hi.size or float(e_hi.mean() - e_lo.mean()) < 0.12:
95 105 return [(0, n)]
106 +
107 + # nperseg 128: bin 7.8 kHz keeps the narrowband gain for real signal widths while the
108 + # 64-sample hop resolves inter-burst gaps down to ~200 samples (256 could not split a
109 + # 300-sample gap -- no column fits inside it).
110 + P, hop, nperseg = _cell_power(x, fs, nperseg=128)
111 + if P.shape[1] < 4: # too few columns to tell bursts from record
112 + return [(0, n)]
113 +
114 + # Per-bin floor at a LOW percentile: tracks coloured noise bin by bin and stays on the
115 + # noise even when a bin is signal-occupied up to ~90% of the time (high-duty trains).

```

```

114 + # A record-filling signal owns its bins' floors entirely -> flat score -> [(0, n)].
115 + floor_f = np.percentile(P, 10, axis=1)[: , None]
116 + r = P / (floor_f + 1e-30)
117 + k = max(3, P.shape[0] // 16)
118 + sc = np.log10(np.sort(r, axis=0)[-k:].mean(axis=0) + 1e-30) # column score (decades)
119 + # A single noise column can spike +0.53 decades over base -- OVERLAPPING the weakest
120 + # real bursts (+0.57). Like the eye, the discriminator is PERSISTENCE, not height: a
121 + # 3-column smooth drops the noise worst-case to 0.31-0.33 while a real burst (>=3
122 + # columns) keeps its level (0.56+) -- the gap the fixed bars below live in.
123 + sc = uniform_filter1d(sc, 3)
124 + # The noise base is the LOWEST MODE of the score histogram (Otsu, as the wideband
125 + # version proved out): fixed quantile anchors cannot serve every duty regime, and
126 + # spread ESTIMATES broke three ways (median blind >50% duty, two-sided MAD inflated by
127 + # skirts, one-sided by the score's left tail). The margins are FIXED instead: a noise
128 + # column's score is distribution-pinned at C = log10((ln(nbins/k)+1)/0.105) whatever
129 + # the noise level, so its fluctuation is a constant of (nbins, k) -- measured worst
130 + # 0.33 over 24 noise records vs 0.56 for the weakest keeper burst.
131 + thr = _otsu(sc, bins=64)
132 + quiet = sc[sc < thr]
133 + if not quiet.size:
134 +     return [(0, n)]
135 + base = float(np.median(quiet))
136 + # No noise-quiet column at all (base far above the pinned C): a continuous signal
137 + # shuffling its spectrum (a 4-tone FSK read as 78 phantom bursts before this guard).
138 + c_noise = float(np.log10((np.log(P.shape[0] / k) + 1) / 0.105))
139 + if base > c_noise + 0.25:
140 +     return [(0, n)]
141 + hi = base + 0.45
142 + lo = base + 0.25
143 + above_lo = sc >= lo
103 144 dif = np.diff(above_lo.astype(np.int8))
104 145 starts = list(np.where(dif == 1)[0] + 1)
105 146 ends = list(np.where(dif == -1)[0] + 1)
106 147 if above_lo[0]:
107 148     starts.insert(0, 0)
108 149 if above_lo[-1]:
150 +     ends.append(sc.size)
151 + runs = [(s, e) for s, e in zip(starts, ends, strict=True) if (sc[s:e] >= hi).any()]
111 152 if not runs:
112 153     return [(0, n)]
122 154 merged = [list(runs[0])]
123 155 for s, e in runs[1:]:
156 + # A real inter-burst gap returns to the score base; a marginal-level burst dips just
157 + # below lo without reaching it and would otherwise shatter into fragments. Merge
158 + # across any valley whose median stays off the base.
159 + valley = sc[merged[-1][1]:s]
160 + if s - merged[-1][1] < 2 or float(np.median(valley)) > base + 0.12:
128 161     merged[-1][1] = e
129 162 else:
130 163     merged.append([s, e])
164 + # Shape gate: a modulated burst has a near-flat raw envelope (peak/mean ~ a few); an
165 + # impulse smear is one spike over noise (ratio ~span). Reject spike-dominated candidates.
136 166 out, spiked = [], 0
167 + for c0, c1 in merged:
168 +     if c1 - c0 < 2: # single-column flicker (noise / impulse smear)
139 169         continue
170 + s, e = max(0, c0 * hop), min(n, (c1 - 1) * hop + nperseg)
171 + if c1 - c0 >= 6:
172 +     # long bursts: trim one hop of smear/quantization slack per edge. The noise
173 +     # tail otherwise fed to the demod sits on a knife edge for dense QAM (58 extra

```

```

174 + # leading samples flipped a 16qam@24dB slice to 64qam lock 18). Short runs keep
175 + # their full span -- trimming there would eat into the burst itself.
176 + s, e = s + hop, e - hop
140 177 seg = pw[s:e]
141 178 if float(seg.max() / (seg.mean() + 1e-30)) > _SPIKE_PAR:
142 179     spiked += e - s
143 180     continue
144 181     out.append((int(s), int(e)))
145 182 # A detection must EXPLAIN the candidate mass the SPIKE gate threw away. If an impulse
183 + # inside the strongest burst killed its candidate, returning the survivors would hand
184 + # analyze() the wrong emitter -- fall back instead. Only spike kills count: the
185 + # single-column drops above are routine flicker filtering.
151 186 covered = sum(e - s for s, e in out)
152 187 if out and covered < 0.6 * (covered + spiked):
153 188     return [(0, n)]
189 + # A continuous signal OWNS its bins' floors and is invisible to the cell path -- but its
190 + # edge transients are not, and once returned [(0, 192)] for a 64k channel whose 16qam
191 + # ran the whole record. If the detections cover almost none of the envelope's high-power
192 + # mass, the cell path missed the main event: return the ENVELOPE's high span instead
193 + # (not the raw record -- a dead tail fed to the demod flipped that 16qam to 64qam).
194 + # 0.1 keeps partial detections (a strong burst next to a lo-only sibling covers ~30%).
195 + if out and covered < 0.1 * int((lp >= e_thr).sum()):
196 +     idx = np.where(lp >= e_thr)[0]
197 +     return [(int(idx[0]), int(idx[-1]) + 1)]
154 198 return out or [(0, n)]
155 199
156 200
157 201 def find_burst(x: np.ndarray, fs: float) -> tuple[int, int]:
158 202     """The most energetic burst of find_bursts."""
159 203     return max(find_bursts(x, fs), key=lambda b: float(np.sum(np.abs(x[b[0]:b[1]]) ** 2)))
160 204
161 205
162 206 # --- carrier estimation -----
163 207
164 208 def est_carrier(
165 209     x: np.ndarray, fs: float, powers: tuple[int, ...] = (2, 4, 8),
166 210     nfft: int = 65536, blocks: int = 4,
167 211 ) -> tuple[float, int, bool]:
168 212     """Blind M-th-power carrier estimate; returns (fc, symmetry, ambiguous). No DC
169 213     nulling: the front-end is DC-blocked, so a peak at DC is a genuine fc=0."""
170 214     freqs = np.fft.fftshift(np.fft.fftfreq(nfft, 1 / fs))
171 215     best_par, best_s, sym = -1.0, None, powers[0]
172 216     for p in powers:
173 217         xp = x ** p
174 218         blk = xp.size // blocks
175 219         segs = [xp[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [xp]
176 220         acc = np.zeros(nfft)
177 221         for s in segs:
178 222             acc += np.abs(np.fft.fftshift(np.fft.fft(s * np.hanning(s.size), nfft)))
179 223         spec = acc / len(segs)
180 224         par = spec.max() / spec.mean()
181 225         if par > best_par:
182 226             best_par, best_s, sym = par, spec, p
183 227
184 228     if best_s is None: # degenerate input (all-zero / no energy): no M-th-power tone exists
185 229         return 0.0, int(powers[0]), True
186 230     ydb = 20 * np.log10(best_s + 1e-20)
187 231     k = int(np.argmax(best_s))
188 232     f_peak = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
189 233     fc = f_peak / sym

```

```

190 234     ambiguous = abs(sym * fc) > 0.4 * fs
191 235     return float(fc), int(sym), bool(ambiguous)
192 236
193 237
194 238 def resolve_alias(x: np.ndarray, fs: float, fc: float, sym: int) -> float:
195 239     """The M-th-power tone wraps mod fs, so fc is only known mod fs/sym. Candidates tile the
196 240     whole band at spacing fs/sym; pick the one whose alias CELL holds the most signal power
197 241     (wrap-aware). Cell-energy beats a spectral centroid at the band edge, where a signal
198 242     straddling +-fs/2 corrupts the two-sided centroid (bpsk fc=0.495*fs picked fc=-5000)."""
199 243     f, pxx = welch(x, fs=fs, nperseg=min(4096, x.size), return_onesided=False)
200 244     p = np.maximum(pxx - np.median(pxx), 0)
201 245     # range must reach +-fs/2 for EVERY sym: |k| up to sym//2 (an old fixed range(-2,3) left
202 246     # 8psk beyond 0.3125*fs with no candidate).
203 247     cands = [fc + k * fs / sym for k in range(-(sym // 2) - 1, sym // 2 + 2)]
204 248     if abs(fc + k * fs / sym) < 0.5 * fs]
205 249     half = fs / (2 * sym) # each candidate owns a fs/sym-wide cell
206 250     return max(cands, key=lambda c: float(p[np.abs(((f - c + fs / 2) % fs) - fs / 2) < half].sum
    l, l,    ()))
207 251
208 252
209 253 def mix(x: np.ndarray, fs: float, fc: float) -> np.ndarray:
210 254     """Downconvert by fc (complex heterodyne)."""
211 255     return x * np.exp(-2j * np.pi * fc * np.arange(x.size) / fs)
212 256
213 257
214 258 # --- symbol rate + rolloff -----
215 259
216 260 def est_baud(
217 261     x: np.ndarray, fs: float, lo: float | None = None, hi: float | None = None,
218 262     blocks: int = 4,
219 263 ) -> tuple[float, float]:
220 264     """Cyclostationary line at the symbol rate in |x|^2; returns (baud, confidence).
221 265     Low confidence flags a weak line (near-zero rolloff); caller decides. Fewer blocks =
222 266     a longer, stronger line (worth trying on a short burst where the default 4 is too weak)."""
223 267     u = np.abs(x) ** 2
224 268     u = u - u.mean()
225 269     blk = u.size // blocks
226 270     segs = [u[i * blk:(i + 1) * blk] for i in range(blocks)] if blk >= 8 else [u]
227 271     nfft = 1 << max(16, int(np.ceil(np.log2(max(s.size for s in segs))))))
228 272     acc = np.zeros(nfft // 2 + 1)
229 273     for s in segs:
230 274         acc += np.abs(np.fft.rfft(s * np.hanning(s.size), nfft))
231 275     spec = acc / len(segs)
232 276     freqs = np.fft.rfftfreq(nfft, 1 / fs)
233 277
234 278     lo = lo if lo is not None else 0.005 * fs
235 279     hi = hi if hi is not None else 0.45 * fs
236 280     band = (freqs >= lo) & (freqs <= hi)
237 281     if not band.any(): # degenerate window (e.g. noise-derived prior): default band
238 282         band = (freqs >= 0.005 * fs) & (freqs <= 0.45 * fs)
239 283     idx = np.where(band)[0]
240 284     k = idx[int(np.argmax(spec[idx]))]
241 285     ydb = 20 * np.log10(spec + 1e-20)
242 286     baud = freqs[k] + _parab(ydb, k) * (freqs[1] - freqs[0])
243 287     conf = spec[k] / (np.median(spec[band]) + 1e-20)
244 288     return float(baud), float(conf)
245 289
246 290
247 291 def occupied_bw(x: np.ndarray, fs: float) -> float:
248 292     """Two-sided occupied bandwidth without a baud prior: threshold 10% of the way

```

```

249 293         from the 25th-percentile floor to the 90th-percentile in-band level, walk
250 294         outward from DC, stop at the first >20-bin below-threshold gap (notch tolerant).
251 295         Returns 0.0 when no band hugs DC (caller must not trust the prior then)."""
252 296     f, pxx = welch(x, fs=fs, nperseg=min(8192, x.size), return_onesided=False)
253 297     floor = np.percentile(pxx, 25)
254 298     order = np.argsort(np.abs(f))
255 299     idx = np.flatnonzero(pxx[order] > floor + 0.10 * (np.percentile(pxx, 90) - floor))
256 300     if idx.size == 0:
257 301         return 0.0
258 302     last = idx[-1]
259 303     return float(2 * np.abs(f[order][last]))
260 304
261 305
262 306 def est_rolloff(x: np.ndarray, fs: float, baud: float) -> float:
263 307     """Occupied-bandwidth band-edge estimate mapped through the v1 rolloff calibration."""
264 308     n = x.size
265 309     nper = min(8192, max(256, n // 8))
266 310     f, pxx = welch(x, fs=fs, nperseg=nper, return_onesided=False)
267 311     f, pxx = np.fft.fftshift(f), np.fft.fftshift(pxx)
268 312     af = np.abs(f)
269 313     inband = np.median(pxx[af < 0.25 * baud])
270 314     noise = np.median(pxx[af > 0.9 * baud])
271 315     thr = noise + 0.10 * (inband - noise)
272 316     band = (af > 0.3 * baud) & (af < 1.0 * baud) & (pxx > thr)
273 317     if not band.any():
274 318         return 0.35
275 319     raw = 2 * af[band].max() / baud - 1
276 320     return float(np.clip(1.5 * raw + 0.026, 0.05, 1.0))
277 321
278 322
279 323 # --- resampling + matched filter -----
280 324
281 325 def to_sps(x: np.ndarray, fs: float, baud: float, sps: int = 4) -> np.ndarray:
282 326     """Rational resample to exactly sps samples/symbol."""
283 327     r = Fraction(sps * baud / fs).limit_denominator(2000)
284 328     return resample_poly(x, r.numerator, r.denominator)
285 329
286 330
287 331 def _rx_rrc(alpha: float, span: int, sps: int) -> np.ndarray:
288 332     """RX matched RRC taps, unit energy, odd length; singularities at t=0 and
289 333     |t|=1/4a patched. Written independently of gen.py's TX shaper (anti-shared-bug)."""
290 334     m = (span * sps) // 2
291 335     t = np.arange(-m, m + 1) / sps
292 336     a = alpha
293 337     num = np.sin(np.pi * t * (1 - a)) + 4 * a * t * np.cos(np.pi * t * (1 + a))
294 338     den = np.pi * t * (1 - (4 * a * t) ** 2)
295 339     h = np.divide(num, den, out=np.zeros_like(t), where=den != 0)
296 340     h[t == 0] = 1 - a + 4 * a / np.pi
297 341     if a > 0:
298 342         s = np.isclose(np.abs(t), 1 / (4 * a))
299 343         h[s] = a / np.sqrt(2) * ((1 + 2 / np.pi) * np.sin(np.pi / (4 * a))
300 344             + (1 - 2 / np.pi) * np.cos(np.pi / (4 * a)))
301 345     return h / np.sqrt(np.sum(h ** 2))
302 346
303 347
304 348 def matched(x: np.ndarray, sps: int, alpha: float, span: int = 10) -> np.ndarray:
305 349     """Apply the RX matched RRC filter."""
306 350     return np.convolve(x, _rx_rrc(alpha, span, sps), "same")
307 351
308 352

```

```

309 353 # --- timing recovery -----
310 354
311 355 def timing(x: np.ndarray, sps: int, block: int = 256, out: int = 1) -> np.ndarray:
312 356     """Block-wise 0erder-Meyr timing recovery; returns `out` samples/symbol
313 357     (1 = decisions, 2 = clock-tracked T/2 stream for the fractionally-spaced eq).
314 358     Per block the spectral-line phase gives the offset; unwrapped across blocks
315 359     to track clock drift, then interpolated per symbol."""
316 360     n = sps
317 361     nsym = x.size // n
318 362     if nsym < 1:
319 363         return x[:0].astype(np.complex128)
320 364     xt = x[:nsym * n]
321 365     p = np.abs(xt) ** 2
322 366     ph = np.exp(-2j * np.pi * np.arange(xt.size) / n)
323 367     bs = block * n
324 368     if xt.size <= bs: # single block: one global fractional offset
325 369         eps = np.full(nsym, -np.angle(np.sum(p * ph)) / (2 * np.pi))
326 370     else:
327 371         nblk = xt.size // bs
328 372         c = (p[:nblk * bs].reshape(nblk, bs) * ph[:nblk * bs].reshape(nblk, bs)).sum(1)
329 373         eps_b = np.unwrap(-np.angle(c)) / (2 * np.pi)
330 374         eps = np.interp(np.arange(nsym), (np.arange(nblk) + 0.5) * block, eps_b)
331 375         pos = (np.arange(nsym * out) // out + np.repeat(eps, out)) * n
332 376         idx = np.arange(xt.size)
333 377         return np.interp(pos, idx, xt.real) + 1j * np.interp(pos, idx, xt.imag)
334 378
335 379
336 380 # --- decision-directed carrier sync -----
337 381
338 382 def ddsync(symbols: np.ndarray, mod: str, alpha: float = 0.05, beta: float = 0.002) -> np.ndarra
339 383     y:
340 384     """Decision-directed PI carrier loop, general across PSK/QAM (never a PSK-only
341 385     Costas loop, which jitters QAM at the correct points). Sequential feedback (each
342 386     phase update depends on the prior decision) -> numba kernel when available."""
343 387     pts = ideal_points(mod)
344 388     z = symbols / np.sqrt(np.mean(np.abs(symbols) ** 2)) # AGC to unit power
345 389     return dd_carrier(np.ascontiguousarray(z, dtype=np.complex128),
        np.ascontiguousarray(pts, dtype=np.complex128), alpha, beta)

```